
accelerate
collaborate
innovate
communicate
palm
PalmSource
Conference 2000



Technology Primer

Gavin Maxwell & JB Parrett
Master Trainers

PalmSource
Conference 2000

Introductions - Who Are We?

- Gavin Maxwell, Master Trainer
  - gavin_maxwell@palm.com
- JB Parrett, Master Trainer
  - jb_parrett@palm.com

Who Are You?

- Developers with C/C++ experience who:
  - Have no Palm development experience,
  - Are itching to get into Palm development, and
  - Want to get the most out of PalmSource 2000



PalmSource Conference 2000

Format and Agenda

- Overview of the Palm OS® and Hardware
- Architecture of a Palm OS solution
- Tools of the trade
- API Overview

- Don’t panic - we will be taking breaks!

Overview of the Palm OS and Hardware

- Palm badged, licensee and OEM devices
- Historical OS Information
- Current OS Implementation

Palm Badged, Licensee and OEM Devices

- Lots, and the list is growing!





Palm Badged, Licensee and OEM Devices

Device	OS	Memory	Heap(K)	Extras
Palm™ Vx	3.5	8MB	256	Slim, rechargeable
Palm™ IIIc	3.5	8 MB	256	Color
Palm™ VIIx™	3.5	8MB	256	Web Clipping
Symbol SPT 1500	3.0	2/4MB	96	Barcode scanner
Symbol SPT 17XX	3.2	2/8MB	128	Barcode, wireless
Kyocera pdQ	3.0	2MB	128	Cell phone
Handspring Visor	3.1	2/8MB	128	Springboard slot
Handspring Prism	3.5	8MB	256	Springboard slot, Color
Sony Clie	3.5	8MB	256	MemoryStick

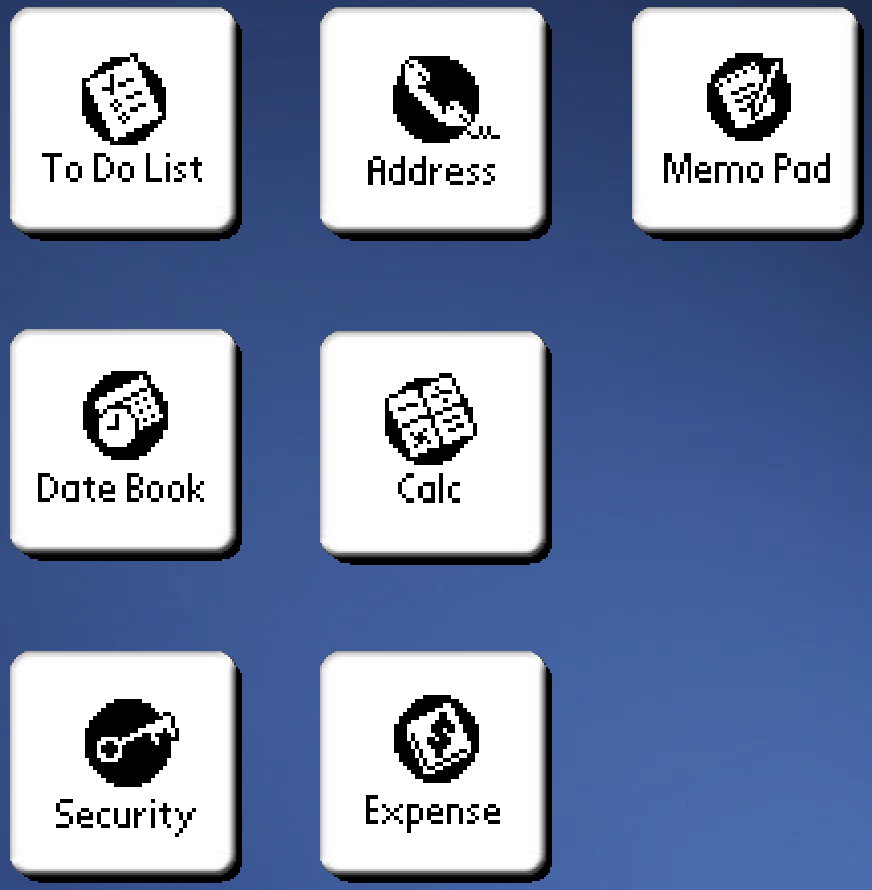
Basic Device Make-up

- Four application buttons
- Scroll buttons
- Silk Screened area (Graffiti)
- On/Off button
- Contrast activation (wheel, button, soft)
- Reset button



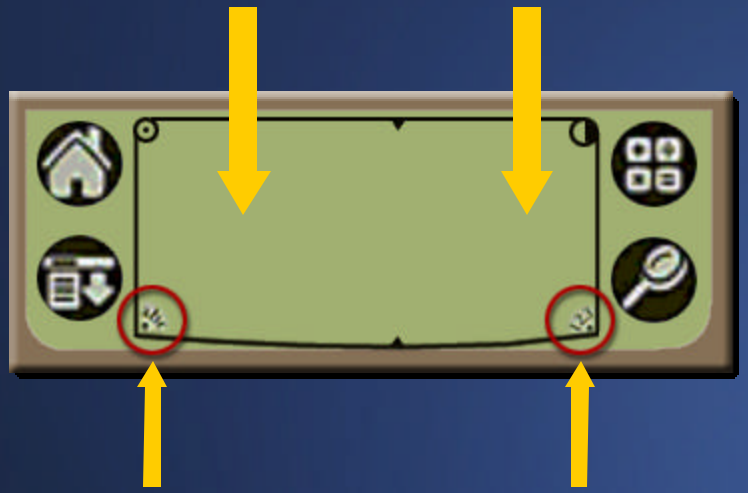
Built-in Applications

- Date Book
- Address Book
- To Do List
- Memo Pad
- Calculator
- Expense
- Security



User Input

- Graffiti input area
  - Letters and Numbers



  - Virtual keyboards



User Input

- Virtual keyboards





Palm OS Information

- Designed for inexpensive handheld devices
  - Small memory footprint
  - Relatively low power processor
  - Low power requirements
  - Small screen
- Simple, elegant, smart
  - Out of Box Experience is key!

Palm OS Overview

- Events, Memory and Databases
- Forms, Form objects and Menus
- Communications, HotSync® and Conduits

Palm OS Overview

Events

- Event driven OS
- Events include:
  - Pen down
  - Form open
  - Key press
  - Control selection etc.
- Low level events can trigger higher level
  - Eg. penDownEvent -> cltEnterEvent

Palm OS Overview

Events

Sample Event Loop

```
Repeat
    Get an event from queue
    Let the system try to handle the event
    If not handled, let the menu manager try
    to handle the event
    If not handled, check for form load
    events and handle them
    If not handled, let form manager try to
    handle the event
Until an appStopEvent
```


PalmSource Conference 2000

Palm OS Overview

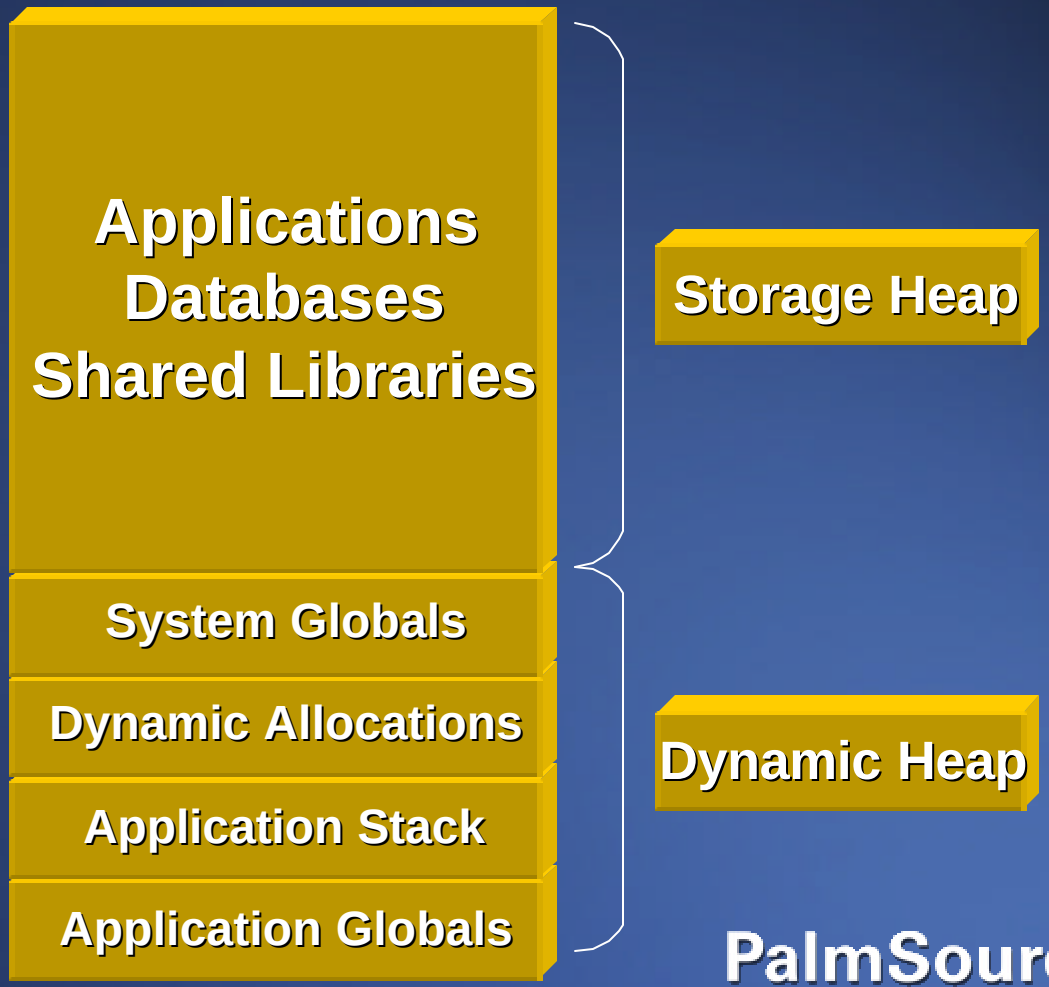
Memory

- Two different types of memory
  - Dynamic
    - System use
    - Current application
  - Storage
    - Applications
    - Persistent data

Palm OS Overview

Memory

- Not to scale!



Palm OS Overview

Memory

- Dynamic heap
  - Divided between System and current application
  - Two types of allocation
    - Relocatable (handles)
      - Lock
      - Use
      - Unlock
    - Non-relocatable (pointers)

Palm OS Overview

Memory

- Relocatable allocations
  - Preferable
  - Allows contiguous free space management
  - Master pointer table



Palm OS Overview

Databases

- Attributes
  - Name
  - Creator Id and Type
  - Application information block
- Collection of records
  - Index, Unique Id
  - Attributes (modified, deleted, category)
  - 64 K maximum size per record

Palm OS Overview

Databases

- Records are simple relocatable blocks
  - It's all RAM isn't it?
  - Storage heap hardware write-protected
  - Specific API to modify storage heap
    - Stops buggy apps trashing other databases!
  - Accessed via handles at runtime
    - References stored within heap as LocalID's though
      - LocalID = offset of record from base address of memory card

Palm OS Overview

Forms

- Forms are application windows
- Only one *active* form
- Forms are containers for form objects
- Modal and modeless forms

Palm OS Overview

Form Objects

- Form objects are user-interface elements
- Rich selection of objects:
  - Buttons
  - Checkboxes
  - Push buttons
  - Tables
  - Text fields
  - Labels
  - Lists
  - Bitmaps
  - User-defined

Palm OS Overview

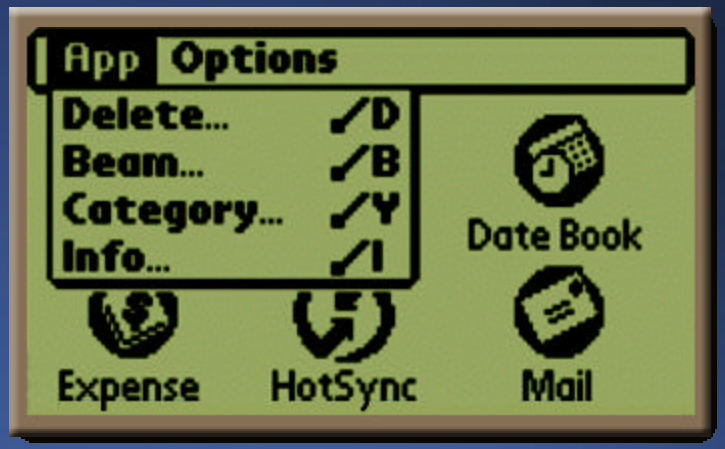
Form Objects

- Forms and form objects stored in resources
  - Very similar to Mac resources
- Specific event generated by FrmHandleEvent receiving low levels event
  - ctlSelectEvent
  - lstSelectEvent
  - ctlRepeatEvent
  - fldEnterEvent

Palm OS Overview

Menus

- A menubar is associated with a form
  - Created at build time
  - API to modify at runtime
- Menubar contains menus
- Menus contain menu items
- Menu event generated upon item selection
  - Direct tap
  - Shortcut



Palm OS Overview

Communications

- Serial
- TCP/IP
  - PPP (over serial or modem)
  - Wireless
    - Symbol 802.11a - Symbol SPT1740 Spectrum 24
    - GSM, CDPD, CDMA etc
- Infrared
  - IrDA

Palm OS Overview

Serial

- Synchronous transmission
  - Receive via polling
- Multiple physical and virtual devices
  - IrCOMM virtual serial device
- Foreground and background operation
  - Keyboard driver

Palm OS Overview

TCP/IP

- Palm API or Berkeley Sockets
  - Your choice (with caveats)
  - Port Berkeley code with minimal or no changes
- Palm OS Emulator TCP/IP re-direction!
  - Great for developing!

Palm OS Overview

Infrared

- IrDA standard
- “Beaming” uses ObEx layer (object exchange)
  - Between Palm’s, some cell phones, other devices

Palm OS Overview

HotSync

- Connectivity options for HotSync
- Conduits
- Synchronization server

Palm OS Overview

HotSync

- Connectivity
  - Serial (cable or Infrared)
  - Modem (wired or wireless)
  - Ethernet (wired or wireless)

Palm OS Overview

Conduits

- Code module called during HotSync process
  - Windows DLL
  - Macintosh Shared Library
- Registered for a particular application
- Example
  - Memo Pad conduit performs two-way sync for Memo Pad application
  - Expense conduit downloads expense records and creates a report
  - Install conduit installs new software on handheld

Palm OS Overview

Conduits

- A conduit can:
  - Read and write records from any database
  - Specify a record using a unique ID or a record index
  - Iterate through only dirty records
  - Manipulate a database (for example, app info block, delete/add records)
  - Create/delete databases

Palm OS Overview

Synchronization server

- HotSync Server
  - Enterprise solution
  - Allows multiple concurrent connections
  - Integration with backend databases via ODBC

Palm OS Overview Q&A

Break

PalmSource Conference 2000

Architecture of a Palm OS Solution

- Application design principles
- Applications and the OS
- PilotMain
  - Startup and shutdown
  - Application preferences
  - The event loop
- Calling an application
  - Occasions when an application might be called
- Backend options

Architecture of a Palm OS Solution

Application Design Principles

- Your application needs to work within the current device design
  - Small
  - Affordable
  - Fast
  - Desktop integration

Architecture of a Palm OS Solution

Application Design Principles

- Device feature: Small
  - Small screen (160 x 160)
  - No fixed keyboard
- Principles
  - Prioritize based on frequency
  - Limit input to necessary tasks
  - Provide command buttons for common multi-step activities

Architecture of a Palm OS Solution

Application Design Principles

- Device feature: Affordable
  - Processor
  - Screen
  - Memory
- Principles
  - Do processor-intensive tasks on the desktop
  - Minimize screen clutter
  - Only necessary features and data

Architecture of a Palm OS Solution

Application Design Principles

- Device feature: Fast
  - ‘Good enough’ text recognition
  - Limits on processing and data manipulation
  - No busy cursor
- Principles
  - Time is measured in fractions of a second
  - Use command buttons
  - Minimize taps

Architecture of a Palm OS Solution

Application Design Principles

- Device feature: Desktop integration
  - Multi-device environment
  - Data gets entered in multiple places
- Principles
  - One button syncing
  - Rely on the desktop for processing information
  - Intelligent data communication between handheld and desktop

Architecture of a Palm OS Solution

Applications and the OS

- Only one application active at any one time
- Palm OS is event driven
- Each application responsible for the event loop
- Application identified by name and creator ID

PalmSource Conference 2000

Architecture of a Palm OS Solution

PilotMain

- Main entry-point
- Responsible for application
  - Startup
  - Event loop
  - Shutdown

Architecture of a Palm OS Solution

PilotMain

```
UInt32 PilotMain( UInt16 launchCode, MemPtr
  launchParams, UInt16 launchFlags )
{
  Err err = 0;
  if ( launchCode == sysAppLaunchCmdNormalLaunch )
  {
       if ( ( err = AppStartApplication() ) == 0 )
       {
            AppEventLoop();
            AppStopApplication();
       }
  }
  return err;
}
```


PalmSource Conference 2000

Architecture of a Palm OS Solution

Application Startup

- Reads application preferences
- Opens databases
- Opens main form
- Other initialization

Architecture of a Palm OS Solution

Application Shutdown

- Writes out application preferences
- Closes databases
- Closes forms

Architecture of a Palm OS Solution

Application Preferences

- PrefGetAppPreferences
  - Creator ID used to identify preferences
  - Pointer to the structure to fill
  - Version number
- PrefSetAppPreferences
  - Creator ID
  - Pointer to structure to store
  - Version number

Architecture of a Palm OS Solution

Application Event Loop

Gives OS managers a chance to handle things

```
static void AppEventLoop(void)
{
   EventType event;

   do
   {
      EvtGetEvent( &event, evtWaitForever );
      if ( !SysHandleEvent( &event ) )
         if (!MenuHandleEvent(0, &event, &error))
            if ( !AppHandleEvent( &event ) )
               FrmDispatchEvent( &event );
   } while ( event.eType != appStopEvent );
}
```

Architecture of a Palm OS Solution

Application Event Loop

- EvtGetEvent
  - Retrieves the next available event from queue
  - Can wait forever, or for a specified period
    - nilEvent

Architecture of a Palm OS Solution

Application Event Loop

- SysHandleEvent
  - Handles all system level events
  - Hard button presses
    - Date Book, Address Book, To Do, Memo Pad
  - Silk screen buttons
    - Launcher, Menu, Calculator, Find
  - Graffiti software

Architecture of a Palm OS Solution

Application Event Loop

- MenuHandleEvent
  - Handles all menu related events
  - Displaying menus
  - Handling menu taps
  - Handling shortcut characters

Architecture of a Palm OS Solution

Application Event Loop

- AppHandleEvent
  - Provided by you!
- Handles the loading and initializing of forms
  - Associating forms with specific event handler
- Your applications first chance to handle an event
- Returns true if event was handled

Architecture of a Palm OS Solution

AppHandleEvent

```
static Boolean AppHandleEvent( EventType *event )
{
    Boolean handled = false;
    if ( event->eType == frmLoadEvent )
    {
        FormPtr frmP = FrmInitForm(
                            event->data.frmLoad.formID );
        FrmSetActiveForm( frm );
        switch( event->data.frmLoad.formID )
        {
            case MainForm:
            FrmSetEventHandler( frmP, FormHandleEvent );
            break;
        }
        handled = true;
    }
    return handled;
}
```


PalmSource Conference 2000

Architecture of a Palm OS Solution

Application Event Loop

- FrmDispatchEvent
  - Calls current form's event handler
  - If event handler returns true nothing further happens
  - Otherwise standard UI functionality occurs

Architecture of a Palm OS Solution

Calling an Application

- Applications are called by the OS
  - After a reset
  - After a sync
  - During a find request
  - When the device receives a beam
  - When an alarm goes off
  - When the system time or date changes
- The application uses launch codes to distinguish between these circumstances

Architecture of a Palm OS Solution

Calling an Application

- Applications can create their own launch codes
  - Inter-application communications
    - Address Book phone list request
- Launch codes tell the application what to do
- The launch flags specify how the application was called
  - Whether application is active
  - Whether application globals are accessible

Architecture of a Palm OS Solution

Calling an Application

- Example
  - When the user does a ‘Find’ a launch code is sent to the current application, followed by every other application on the device
  - Only the current running application can access it’s global variables

Architecture of a Palm OS Solution

Backend Options

- Custom conduit
- HotSync Server
- Sybase Ultralite
- Others

Architecture of a Palm OS Solution

Backend Options

- Custom conduit
  - Written using Conduit Development Kit (C/C++, Java)
  - No real limitations on what a conduit can do
  - Single user

Architecture of a Palm OS Solution

Backend Options

- HotSync Server
  - Enterprise solution
  - Multiple concurrent users
  - HSS Conduit can talk to ODBC data source

Architecture of a Palm OS Solution

Backend Options

- Sybase Ultralite
  - Handheld and server solution
  - Customized handheld SQL database engine
    - DB engine created at compile time
  - Synchronize with Sybase database
    - Any other ODBC data source also

Architecture of a Palm OS Solution

Backend Options

- Others
  - XTNDConnect
  - WaveSync
  - AvantGo Server
  - IntelliSync Anywhere



Architecture of a Palm OS Solution Q&A

Break

PalmSource Conference 2000

Tools of the Trade

- CodeWarrior®
- Palm OS Emulator
- Palm OS Sources
- PRC-Tools
- Satellite Forms
- Other options

Tools of the Trade

CodeWarrior

- CodeWarrior IDE
- C/C++ Compiler
- Constructor (resource editor)
- PalmRez (resource compiler)
- Ships with
  - Palm OS SDK
  - Symbol SDK
  - Handspring SDK

Tools of the Trade

Palm OS Emulator - POSE

- Emulates range of Palm hardware
  - Requires a ROM file to run
    - Debug ROMs from Palm
    - Non-debug ROMs from real device or Palm
  - Including HotSync, serial and TCP/IP
- Windows, Macintosh, Linux versions



Tools of the Trade

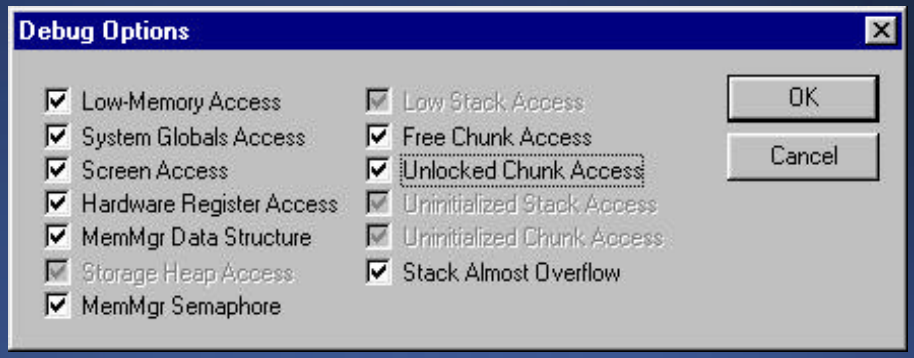
Palm OS Emulator - POSE

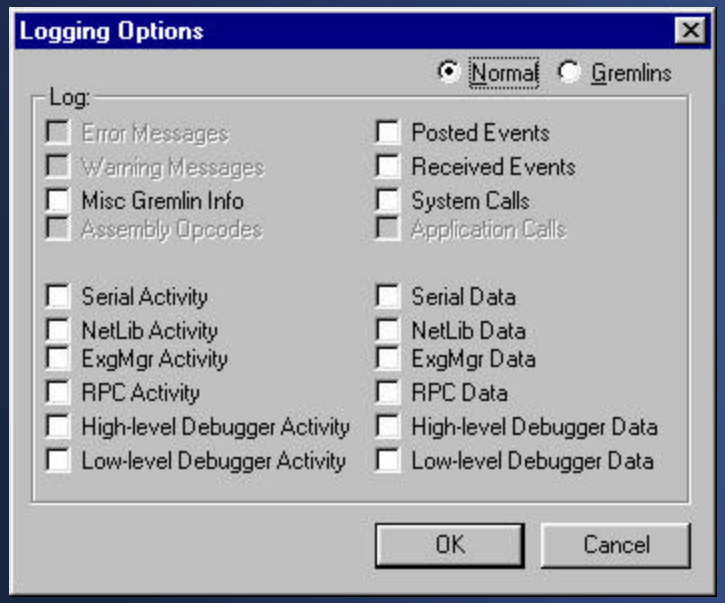
- Allows:
  - Sanity checking
  - Performance measurement
  - Automated testing
  - Serial and TCP/IP debugging
  - Conduit debugging
- Open Source
- Frequent updates:
  - www.palmos.com/dev/tech/tools/emulator

Tools of the Trade

Palm OS Emulator - POSE

- Debugging
- Logging





Tools of the Trade

CodeWarrior and POSE

Walkthrough

Tools of the Trade

Palm OS Sources

- Available under a no-fee license to registered solution providers
  - www.palmos.com/dev/
- Useful when debugging
  - Does not allow stepping through sources though
- Don’t **do** what the ROM does!
- Don’t **depend** on how the sources work!

Tools of the Trade

PRC-Tools

- Freeware C/C++ Development
  - Windows and Unix
- Command-line tools using modified version of GCC (GNU C Compiler)
- PilRC, a text-based resource compiler
- GUI front-ends exist that make development easier
- Supported by Palm
  - www.palmos.com/dev/tech/tools/gcc

Tools of the Trade

Satellite Forms

- High-level forms generator
  - Direct support for tables
  - Visual Basic-like scripting language for customization
  - Supports bar-coding, IR, serial and TCP/IP communications
- Applications require a runtime engine on the handheld
- Extensible with plug-ins written in C

Tools of the Trade

Satellite Forms

- From Puma Technology
  - <http://www.pumatech.com>
- Windows only
- Limited functionality demo available

Tools of the Trade

Satellite Forms

- From Puma Technology
  - <http://www.pumatech.com>
- Windows only
- Limited functionality demo available

Tools of the Trade

Other options

- Pendragon Forms
- Java 2 Micro Edition
- CASL

Tools of the Trade

Pendragon Forms

- Form-generating application
  - Runs on Windows
  - Can generate forms directly from a database
  - Limited scripting support
- Runtime engine on Palm
- Conduit provides synchronization with desktop database

Tools of the Trade

J2ME

- Small, fast Java Virtual Machine (KVM)
- Library of APIs that are specialized for each type of device
- Profile
  - Minimum set of API for kind of device
  - Spec of Java VM functions required for the API

Tools of the Trade

CASL

- IDE with support for CASL language
- Compiles to byte-code which runs on Windows or on Palm OS
- Requires runtime engine on Palm device



Tools of the Trade Q&A

Break

PalmSource Conference 2000

API Overview

- Memory
- Forms
- Controls
- Databases

API Overview

Memory Manager

- Types of Memory
- Memory Allocations
- Memory Manager APIs
- Tips

PalmSource Conference 2000

API Overview

Types of Memory

- ROM (512K to 4MB) contains
  - OS
  - Built-in applications
  - Default databases
  - ROM is actually flash (for some devices)
- RAM (128K to 8MB) contains
  - Add-on applications and Preferences
  - User data
  - Run time storage—Dynamic Memory

API Overview

Types of Memory

System Resources	*OS 3.5*	*OS 3.0*	*OS 2.0*
System Globals	40KB	6KB	2.5KB
System Dyn. Allocation (TCP/IP, IrDA, etc.)	>50KB	50KB	47KB
App Stack (Call Stack and Local Variables)	3.5KB (default)	3.25KB (default)	2.5KB
Remainder (Application Globals dyn Allocation)	<162.5KB	35.75KB	12KB
Total Dynamic Memory	256KB	96KB	64KB

API Overview

Memory Allocations

- Memory Chunks
  - Allocated within heaps
  - Relocatable (Handles)
  - Nonrelocatable (Pointers)
- Using Handles
  - Lock to obtain pointer to data
  - Use
  - Unlock
- Lock count increments up to 14

API Overview

Memory Allocations

- Allocate/Deallocate
  - `p = MemPtrNew( size )`
  - `MemPtrFree( p )`
  - `h = MemHandleNew( size )`
  - `MemHandleFree( h )`
- Lock/Unlock
  - `p = MemHandleLock( h )`
  - `MemHandleUnlock( h )`
  - `MemPtrUnlock( p )`

API Overview

Memory Manager API

- Size
  - `i = MemHandleSize( h )`
  - `i = MemPtrSize( p )`
  - `MemHandleResize( h, size )`
  - `MemPtrResize( p, size )`
- Heap Information
  - `MemNumHeaps`
  - `MemNumCards`
  - `MemHeapID`
  - `MemHeapFreeBytes`
  - `MemHeapSize`

API Overview

Memory Tips

- Use Pointers:
  - For data that will be used for a short time
  - Or for its entire lifetime
- Use Handles:
  - For data that will be resized (entered data)
  - For data used occasionally during its lifetime

API Overview

Memory Tips

- There is not much space
  - It is easy to run out
- Check result of every memory allocation
  - **`MemPtrNew`**
  - **`MemHandleNew`**
- Check result of every memory resize
  - **`MemPtrResize`**
  - **`MemHandleResize`**

API Overview

Forms

- Form Attributes
- Form Events
- Form API

API Overview

Form Attributes

- Attributes of every form:
  - **Title** The name of the form
  - **Modal** if true, the view is drawn modally
  - **Save Behind** Determines if contents behind a form are saved automatically
  - **Help ID** Set to the ID of a string resource
  - **Menu bar ID** Set the ID of a Menubar
  - **Default button ID** Used for alerts. The ID of the button that dismisses the dialog if the app is quit

API Overview

Form Events

- **`frmLoadEvent`**
- **`frmOpenEvent`**
- **`frmUpdateEvent`**
- **`frmCloseEvent`**
- **`frmSaveEvent`**

API Overview

Form Events

- frmLoadEvent
  - A form needs to be loaded
  - Your code (AppHandleEvent) must:
    - Initialize the form
    - Make it active
    - Install an event handler
- frmOpenEvent
  - A form is being opened
  - The event handler must draw the form

API Overview

Form Events

- frmUpdateEvent
  - A form is being redrawn
  - Default action redraw form with FrmDrawForm
- frmCloseEvent
  - A form is being closed
  - Default action is to erase and close the form
- frmSaveEvent
  - A form should save its data
  - Sent prior to some actions (e.g., a Find)

API Overview

Form Manager API

- FrmAlert displays an alert and returns when one of the alert's buttons is pressed (or if the user switches to another application)
  - **`buttonHit = FrmAlert( alertID )`**
  - **`alertID`** is the resource ID of the alert to display
  - **`buttonHit`** is the index of the button pressed
- If the user switches to another application, the return result is the defaultButtonID (often that of the cancel button)

API Overview

Form Manager API

- FrmCustomAlert is similar to FrmAlert but replaces occurrence of ^1, ^2 and ^3 in the alert text with string1, string2 and string3
  - **`buttonHit = FrmCustomAlert( alertID, string1, string2, string3 )`**
  - If there is a ^1, ^2 or ^3 in the alert text, make sure you pass a corresponding string

API Overview

Form Manager API

- To switch between forms use:
  - FrmGotoForm(formID)
    - Posts frmCloseEvent for the current form
    - Posts frmLoadEvent for the specified form
  - FrmPopupForm(formID)
    - Doesn't close the current form
    - Posts frmLoadEvent for the specified form
    - Rare as it uses more memory
  - FrmReturnToForm(formID)
    - Closes the current form
    - Makes a form whose ID is formID the current form
    - Used to return from a form opened with FrmPopupForm

API Overview

Form Manager API

- To retrieve the current active form use FrmGetActiveForm
  - **`FormPtr frmP = FrmGetActiveForm();`**
  - Often used in form event handlers to obtain the current form.

API Overview

Controls

- Accessing controls
- Setting values
- Control selection
- Dynamic controls

API Overview

Accessing Controls

- To retrieve object index of a control use FrmGetObjectIndex
  - **`objectIndex = FrmGetObjectIndex( form, objectID )`**
  - Used often
- To retrieve the resource ID of a control use FrmGetObjectID
  - **`objectID = FrmGetObjectID( form, objectIndex )`**
  - Used rarely

API Overview

Accessing Controls

- To retrieve a pointer to a control object use FrmGetObjectPtr
  - **`myObject = FrmGetObjectPtr( form, objectIndex )`**
  - You must cast to correct type (for example: ListPtr, ControlPtr, etc.)
- Showing/Hiding Objects
  - **`FrmShowObject( form, objectIndex )`**
  - **`FrmHideObject( form, objectIndex )`**

PalmSource Conference 2000

API Overview

Setting Values

- To change the text displayed in a control use CtlSetLabel
  - **`CtlSetLabel( controlPtr, newTextP )`**
  - Stores newTextP pointer in control
  - newTextP pointer must remain valid while the control is displayed
  - Redraws the control label

API Overview

Setting Values

- Boolean values
  - For checkboxes, push-buttons, and repeating buttons
  - **`Boolean on = CtlGetValue( controlPtr )`**
    - 0 = off, non-zero = on
  - **`CtlSetValue( controlPtr, trueOrFalse )`**
    - Sets the value of the pointer on/off
      - 0 = off, non-zero = on

PalmSource Conference 2000

API Overview

Control Selection

- To set and get the control selection for a push button group use
  - **`objIndex = FrmGetControlGroupSelection ( formPtr, groupNumber )`**
    - Use this call to find out which control in a group is selected
  - **`FrmSetControlGroupSelection( formPtr, groupNumber, controlID )`**
    - Use this calls to set which control in a group is selected

API Overview

Dynamic Controls

- Post OS 3.0 routines allows the creation of dynamic form objects
- Example:
  - **`newLabelP = FrmNewLabel( &formP, newLabelID, "my label text”, xCoord, yCoord, desiredFontID )`**
  - The form and every form object pointer on the form may be relocated after this call, hence the &formP

API Overview

Database Manager

- Introduction
- Memory Manager API
- Tips

API Overview

Introduction

- Permanent data resides in storage heap
- Organized into
  - Databases - simple collection of records
  - Records - relocatable chunk of memory
    - Maximum size of slightly less than 64Kb
- Must use Data Manager API to modify storage heap!
  - Hardware write-protected to protect heap integrity
- OS has always supported multiple cards

API Overview

Data Manager API

- Databases
- Records

API Overview

Databases

- Opening
- Creating
- Closing

API Overview

Opening Databases

- Open a database by type and creator

```
DmOpenRef gDB;
gDB = DmOpenDatabaseByTypeCreator(
  ‘DATA’, ‘CRID’, mode );
```

- Modes
  - dmModeReadWrite
  - dmModeReadOnly
  - dmModeWrite
  - dmModeExclusive
  - dmModeShowSecret
  - dmModeLeaveOpen

API Overview

Creating Databases

- To create a database use DmCreateDatabase:
  - **`Err DmCreateDatabase( UInt16 cardNo, const Char *nameP, UInt32 creator, UInt32 type, Boolean resDB )`**
  - nameP - The name of the database; it must be unique
  - Creator - A unique creator ID registered at Palm
  - Type - To distinguish between multiple databases with different types
- Strategy for database naming:
  - Database name-creator code
  - E.g. MyDatabase–CRID

API Overview

Creating Databases

```
// Find database. If it doesn’t exist, create it.
gDB = DmOpenDatabaseByTypeCreator('DATA',’CRID', mode );
if ( !gDB )
{
  err = DmCreateDatabase( 0, ”MyDatabase-CRID", ’CRID',
       'DATA', false );
  if ( err )
       return err;
  gDB = DmOpenDatabaseByTypeCreator( 'DATA', ’CRID',
       mode );
  if ( !gDB )
       return DmGetLastErr();
  // code to initialize database
}
```

API Overview

Closing Databases

- Close a database with:
  - `err = DmCloseDatabase( gDB );`
- Don’t leave databases open unnecessarily:
  - They use about 50 bytes of space in the dynamic heap
- When you close a database, records that are locked or busy remain that way

API Overview

Records

- A record is a relocatable block in memory
  - Accessed via a handle
- You define the structure/content of a record
- Each record in a database can be a different size
- Maximum length of a record is slightly less than 64K

API Overview

Records

- Reading records
- Creating records
- Modifying records
- Deleting records

PalmSource Conference 2000

API Overview

Reading Records

- MemHandle DmQueryRecord(DmOpenRef db, UInt16 recordNum);
- Lock it, read from it, unlock it.

```
DrawNthMemo(UInt16 n, RectanglePtr r)
{
    MemHandle h = DmQueryRecord(gMemoDB, n);
    if (h)
    {
        MemPtr p = MemHandleLock(h);
        WinDrawChars(p, StrLen(p), xPos, yPos);
        MemHandleUnlock(h);
    }
}
```

API Overview

Creating Records

- Use DmNewRecord to create a new record:
  - `UInt16 recordIndex = dmMaxRecordIndex;`
  - `myRecordHandle = DmNewRecord( gDB, &recordIndex, recordSize )`
- To add a record to the beginning of a database use 0 as the record index
- To add a record to the end use dmMaxRecordIndex
- Call DmReleaseRecord when finished with the record

API Overview

Modifying Records

- Use DmGetRecord to modify a record:
  - This marks a record as busy (as does DmNewRecord)
- Use DmWrite or DmSet to write to the record
- You can't just write to the pointer because the storage area is write protected
- Call DmReleaseRecord when you are done

API Overview

Modifying Records

- There are two ways to modify a record
  - Modify the entire structure
    - Easier
  - Modify only the changed field
    - Writes less data

API Overview

Modifying Records

- Example of the first approach:

```
typedef struct
{
  UInt16 field1;
  Char field2;
} MyStruct;
MyStruct s;

// set fields of s
err = DmWrite( myPointer, 0, &s,
      sizeof(s) );
```

PalmSource Conference 2000

API Overview

Modifying Records

- Example of the second approach:

```
typedef struct
{
  UInt16 field1;
  Char field2;
} MyStruct;
MyStruct s;

Char aChar = 'x';

err = DmWrite( myPointer, OffsetOf(
      MyStruct, field2 ), &aChar,
      sizeof( aChar ) );
```

API Overview

Deleting Records

- Deleting records can be tricky
  - Must take conduits and desktop into consideration
- Possible deletion scenarios
  - A user deletes a just-created record
    - Since there is no record on the desktop, just delete it
  - A user deletes an existing record
    - Hide the record, delete chunk, move header to the end of the database, and mark it for deletion on the desktop
  - A user deletes a record and archives it
    - Hide the record, move it to the end of the database, and mark it for archiving on the desktop
  - Implicit deletion
    - All the data is removed leaving an empty record

API Overview

Deleting Records

- Use DmDeleteRecord for a deleted record
  - The memory chunk is freed
  - The unique ID and attributes are maintained
  - The delete bit is set
- Use DmArchiveRecord for an archived record
  - Memory chunk, unique ID and attr's are maintained
  - The delete bit is set
- Use DmRemoveRecord to completely remove a record
  - The entry in the database header is removed
  - The memory chunk is freed

API Overview

Deleting Records

- To move deleted and archived records to the end of the database use this logic:

```
if (isNew && !gSaveBackup)
  DmRemoveRecord(gDB, recordNumber); //remove all traces
else {
  if (gSaveBackup) //need to archive it on PC
       DmArchiveRecord(gDB, recordNumber);
  else
       DmDeleteRecord(gDB, recordNumber);
  // Deleted records are stored at the end
  DmMoveRecord(gDB, recordNumber, DmNumRecords(gDB));
}
```

API Overview

Database Tips

- DmOpenDatabaseByTypeCreator expects type first, but DmCreateDatabase expects creator first
- Database names must be unique
  - End them with a hyphen and your 4-character creator
- Make the creator of your database the same as the creator of your application
  - The database will be automatically deleted when the user deletes the application

API Overview

Database Tips

- For large amounts of data larger (>64Kb)
  - Coalesce multiple records together yourself
  - Use the File Streaming API
  - Use Feature Manager



API Overview

Q&A

PalmSource

Conference 2000


accelerate
collaborate
innovate
communicate
palm
PalmSource
Conference 2000